

Raspberry Pi

Comunicaciones I2C, SPI, 1-Wire, UART

Introducción

Unidades de información

Antes comenzar a trabajar con las comunicaciones en la Raspberry Pi, es importante mencionar algunos conceptos básicos, el primero son las unidades de información.

Unidad	Descripción
Bit	Con un bit se puede representar sólo un valor de dos posibles valores diferentes, 0 o 1, falso o verdadero
Nibble	Un nibble es una colección de 4 bits. un nibble se presenta un número BCD y también puede representar un dígito hexadecimal
Byte	Un byte consta de 8 bits. Las referencias hacia cierta localidad de memoria en todos los microprocesadores nunca es menor que un byte
Palabra	Una palabra es un grupo de 16 bits. Una palabra se puede dividir en 2 bytes llamados igualmente de bajo y alto orden. También una palabra puede considerarse como un grupo de 4 nibbles.

Introducción

Unidades de información

A continuación mostramos el rango de valores que se pueden obtener con un Byte y con una Palabra.

Unidad	Rango de valores sin signo	Rango de Valores con signo
Byte	0 - 255	-128 - 127
Palabra	0 – 6,535	-32,768 - 32,767

Introducción

Complemento a Dos

El método de complemento a 2 en aritmética es comúnmente usada en computadoras para manipular números negativos.

1. El primer paso consiste en aplicar un NOT al numero binario.
2. Una vez realizado esto, sumamos un 1 al bit menos significativo.

Ejemplo:

```
62,080 = 1111 0010 1000 0000
PASO 1) NOT(1111 0010 1000 0000) = 0000 1101 0111 1111
PASO 2) 0000 1101 0111 1111
        +                   1
         0000 1101 1000 0000 = -3,456
```

Introducción

Operadores a nivel de bits en Python

Operador	Descripción	Ejemplo
&	and	r = 3 & 2 # r es 2
|	or	r = 3 | 2 # r es 3
^	xor	r = 3 ^ 2 # r es 1
~	not	r = ~3 # r es -4
<<	Desplazamiento izq.	r = 3 << 1 # r es 6
>>	Desplazamiento der.	r = 3 >> 1 # r es 1

Introducción

Representación de números Python

Decimal	Hexadecimal	Octal	Binaria
62080	0xf280	0o171200	0b1111001010000000

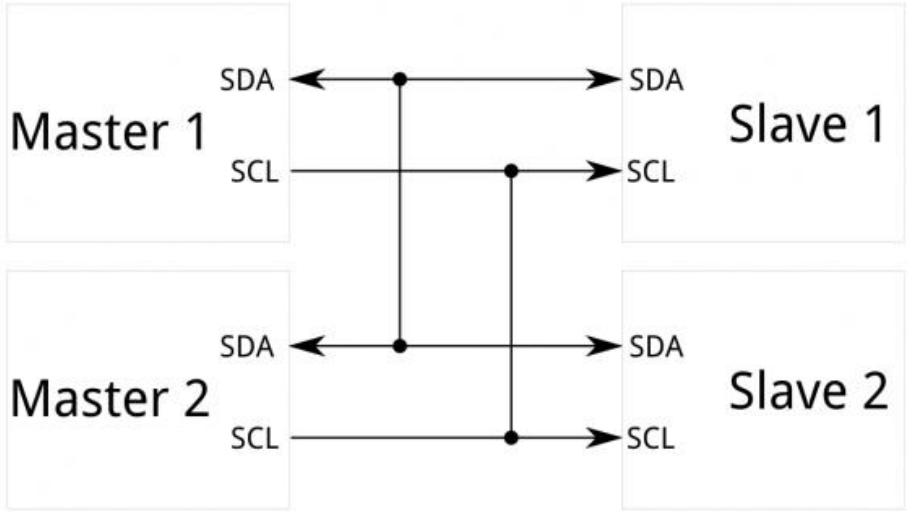
```
>>> hex(62080) #convierte un número a base Hexadecimal
'0xf280'
>>> bin(62080) #convierte un número a base Binaria
'0b1111001010000000'
>>> oct(62080) #convierte un número a base Octal
'0o171200'
>>> int(0xf280) #convierte un número a base Decimal
62080
>>>
```

I2C

Descripción

El protocolo I2C permite que múltiples dispositivos "esclavos" se comuniquen con uno o más dispositivos "maestros“, con el uso de 2 hilos.

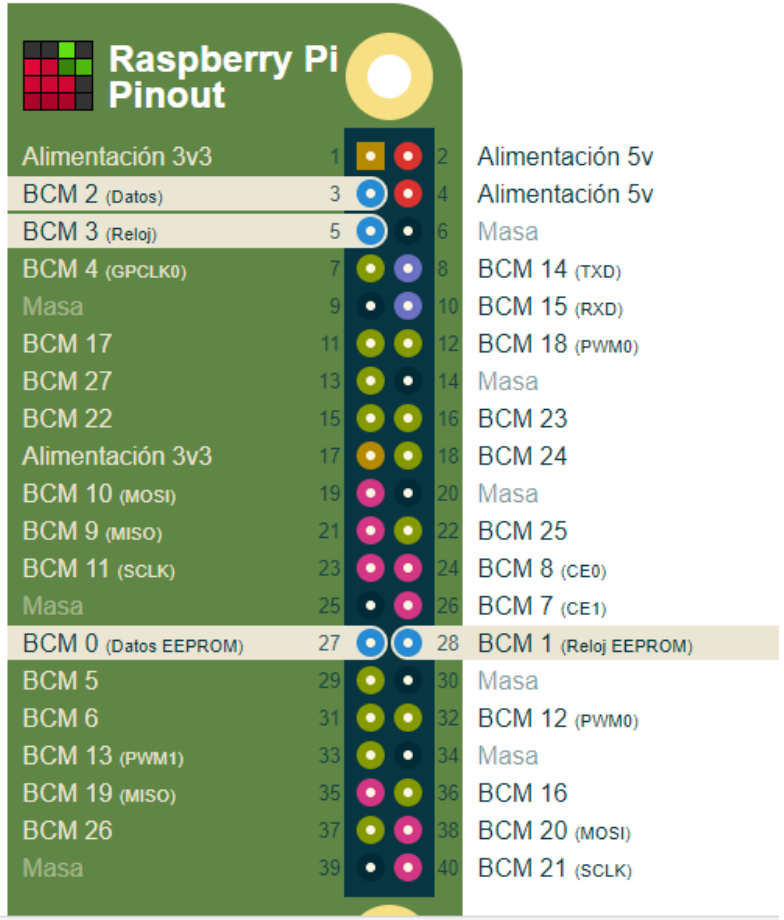
- SDA, medio donde viaja la información.
- SCL, medio por donde viaja la señal de reloj.



python + = Raspbian

I2C

Pines



python

+ = Raspbian

I2C

Habilitar

Para habilitar el Uso de I2C en nuestra Raspebrry es necesario realizar los siguientes pasos:

1. Ingresar a `raspi-config`
2. Seleccionamos la opción “5 Interfacing options”
3. Seleccionamos la opción “P5 I2C”
4. Nos preguntara si queremos habilitar la comunicación I2C y responderemos que si.
5. Nos confirma que ha sido habilitado la conexión I2C y salimos de raspi-config.
6. Reiniciamos nuestra Raspberry Pi.

I2C

Librerías y/o Paquetes Necesarios

Para hacer uso de I2C desde Raspbian es necesario hacer uso del paquete i2c-tolos, lo instalamos con al siguientes instruccuion.

```
sudo apt-get install i2c-tools
```

De igual manera en Python es necesario instalar la librería smbus.

```
sudo apt-get install python-smbus python3-smbus
```

I2C

Ejemplos

A continuación veremos como trabajar con I2C desde la línea de comandos. Lo primero que podemos hacer es descubrir qué interfaces I2C tenemos disponibles:

```
pi@raspi_eder:~ $ i2cdetect -l
i2c-1   i2c             bcm2835 I2C adapter                     I2C adapter
```

I2C

Ejemplos

Solo nos aparece una interfaz I2C (i2c-1). Ahora podemos mirar qué dispositivos hay conectados en el bus.

```
pi@raspi_eder:~ $ i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- 68 -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

I2C

Ejemplos

Ya podemos leer y escribir en cualquiera de los dispositivos I2C conectados. Para leer podemos utilizar `i2cget` y para escribir podemos usar `i2cset`:

```
pi@raspi_eder:~ $ i2cget -y 1 0x68 117
0x68
```

Hemos leído el registro 117 del dispositivo con dirección 0x68 del bus i2c-1

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
73	115	FIFO_COUNTL	R/W	FIFO_COUNT[7:0]							
74	116	FIFO_R_W	R/W	FIFO_DATA[7:0]							
75	117	WHO_AM_I	R	-	WHO_AM_I[6:1]						-

I2C

Ejemplos

Vamos a leer el registro 107, para detectar en que modo esta nuestro MPU6050,

```
pi@raspi_eder:~ $ i2cget -y 1 0x68 107
0x40
```

Eso significa que está puesto a uno el bit 6, que corresponde con el modo SLEEP. El dispositivo arranca en modo dormido para no consumir batería, tenemos que despertarlo quitando ese bit:

```
pi@raspi_eder:~ $ i2cset -y 1 0x68 107 0
```

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
6B	107	PWR_MGMT_1	R/W	DEVICE _RESET	SLEEP	CYCLE	-	TEMP_DIS	CLKSEL[2:0]		



I2C

Ejemplos

Vamos a leer la última medida de la temperatura. La medida está en los regstros 65 y 66, pero si leemos los dos registros de forma independiente podemos estar leyendo parte de la temperatura de una medida y otra parte de otra medida. Por eso hay que leer los dos registros de golpe, haciendo una transferencia de 16 bits (modo palabra):

```
pi@raspi_eder:~ $ i2cget -y 1 0x68 65 w
0x80f2
```

I2C

Ejemplos

Ya tenemos lecturas de temperatura. Pero es muy importante interpretarlas bien. I2C es un protocolo orientado a bytes. Se transfiere byte a byte. Al transferir una palabra por I2C el ordenador entiende que primero llega el byte bajo y luego el alto. Este convenio se llama *little-endian* y es el dominante hoy en día. Sin embargo si vemos el mapa de registros veremos que la dirección 65 corresponde al byte alto y el 66 al byte bajo. ¡Por tanto los bytes están cambiados! La lectura hay que interpretarla como 0xf280.

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
41	65	TEMP_OUT[15:8]							
42	66	TEMP_OUT[7:0]							

I2C

Ejemplos

Vamos a ver cómo podemos hacerlo con la shell:

```
pi@raspi_eder:~ $ TEMP=$(i2cget -y 1 0x68 65 w)
pi@raspi_eder:~ $ echo $TEMP
0x80f2
pi@raspi_eder:~ $ TEMP2=$(echo "0x${TEMP:4:2}${TEMP:2:2}")
pi@raspi_eder:~ $ echo $TEMP2
0xf280
```

Vamos a hacer la transformación completa a una temperatura en grados Celsius:

```
pi@raspi_eder:~ $ echo "$(($TEMP2 - 0x10000))/340 + 36.53" | bc -l
26.365294117647058823
```

I2C

Ejemplos

```
0xf280  =  62,080
0x10000 =  65,536
0xf280 - 0x10000 = 62,080 - 65,536
                 =  -3,456 / 340
                 =  -10.16470588235294 + 36.53
                 =  26.36529411764706
```

The temperature in degrees C for a given register value may be computed as:

Temperature in degrees C = (TEMP_OUT Register Value as a signed quantity)/340 + 36.53

I2C

Ejemplos

```
import smbus
import time
power_1 = 0x6b
power_2 = 0x6c

def read_word(dir):
    high = bus.read_byte_data(direccion, dir)
    low = bus.read_byte_data(direccion, dir+1)
    val = (high << 8) + low
    return val

def read_word_2c(dir):
    val = read_word(dir)
    if(val >= 0x8000):
        return  -((65535 - val)+1)
    else:
        return val
```

```
bus = smbus.SMBus(1)
direccion = 0x68
bus.write_byte_data(direccion, power_1, 0)
try:
    while True:
        temp = read_word_2c(0x41)
        print(temp)
        print("Temp: ", temp / 340 + 36.53)
        time.sleep(0.5)
except KeyboardInterrupt:
    print("Programa   finalizado   por   el
usuario")
finally:
    print("Salida con exito")
```

I2C

Ejemplos

La orden i2cdump permite leer de una ráfaga un conjunto de registros. Leemos de golpe desde el registro 61 hasta el 72. Corresponden a las lecturas de (por orden):

```
pi@raspi_eder:~ $ i2cdump -y -r 59-72 1 0x68 b
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
30:                            d1 28 f7 2c 25             ?(?,%
40: d4 f2 90 fe e7 00 d5 ff dd                          ?????.?.?
```

Acelerómetro.

- Aceleración en X: 0xd128
- Aceleración en Y: 0xf72c
- Aceleración en Z: 0x25d4

Temperatura: 0xf290.

Giróscopo:

- Rotación en X: 0xfee7
- Rotación en Y: 0x00d5
- Rotación en Z: 0xffdd

I2C

Ejemplos

Addr (Hex)	Addr (Dec.)	Register Name	Serial I/F	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
3B	59	ACCEL_XOUT_H	R	ACCEL_XOUT[15:8]							
3C	60	ACCEL_XOUT_L	R	ACCEL_XOUT[7:0]							
3D	61	ACCEL_YOUT_H	R	ACCEL_YOUT[15:8]							
3E	62	ACCEL_YOUT_L	R	ACCEL_YOUT[7:0]							
3F	63	ACCEL_ZOUT_H	R	ACCEL_ZOUT[15:8]							
40	64	ACCEL_ZOUT_L	R	ACCEL_ZOUT[7:0]							
41	65	TEMP_OUT_H	R	TEMP_OUT[15:8]							
42	66	TEMP_OUT_L	R	TEMP_OUT[7:0]							
43	67	GYRO_XOUT_H	R	GYRO_XOUT[15:8]							
44	68	GYRO_XOUT_L	R	GYRO_XOUT[7:0]							
45	69	GYRO_YOUT_H	R	GYRO_YOUT[15:8]							
46	70	GYRO_YOUT_L	R	GYRO_YOUT[7:0]							
47	71	GYRO_ZOUT_H	R	GYRO_ZOUT[15:8]							
48	72	GYRO_ZOUT_L	R	GYRO_ZOUT[7:0]							

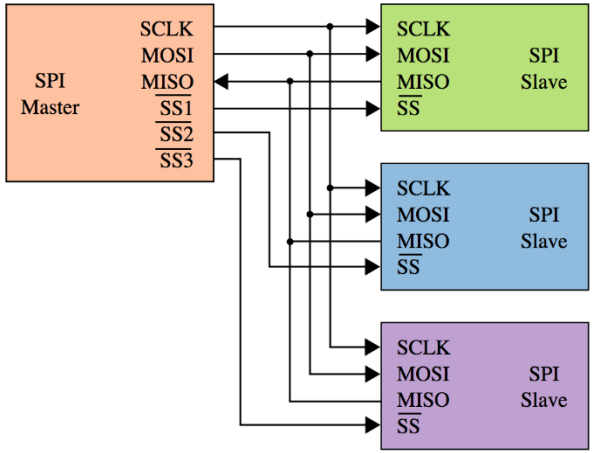


SPI

Descripción

Dentro de este protocolo se define un maestro que será aquel dispositivo encargado de transmitir información a sus esclavos. El SPI es un protocolo síncrono que trabaja en modo full duplex. Hace uso de 4 hilos.

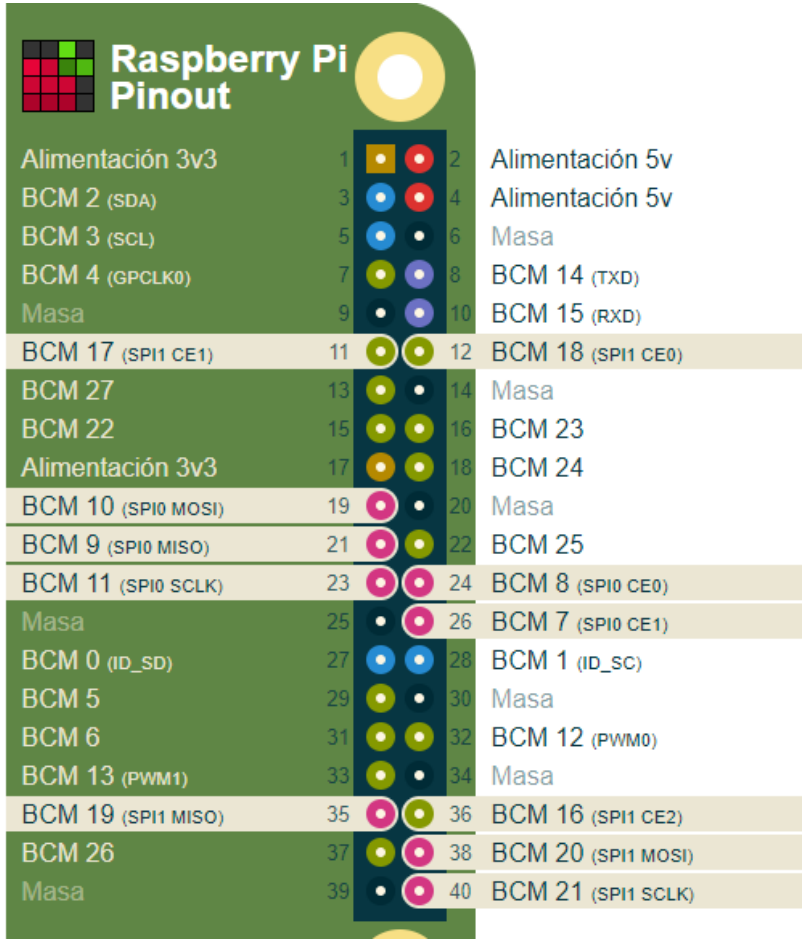
- Master Out Slave In (MOSI)
- Master In Slave Out (MISO)
- Señal de reloj compartida (CLK)
- CEn (Chip Enable). Una o varias señales de selección de destino activas a nivel bajo.



python + = Raspbian

SPI

Pines



python + = Raspbian

SPI

Habilitar

Para habilitar el Uso de I2C en nuestra Raspebrry es necesario realizar los siguientes pasos:

1. Ingresar a `raspi-config`
2. Seleccionamos la opción “5 Interfacing options”
3. Seleccionamos la opción “A6 SPI”
4. Nos preguntara si queremos habilitar la comunicación SPI y responderemos que si

SPI

Ejemplos

A continuación veremos hacer uso del protocolo ISP



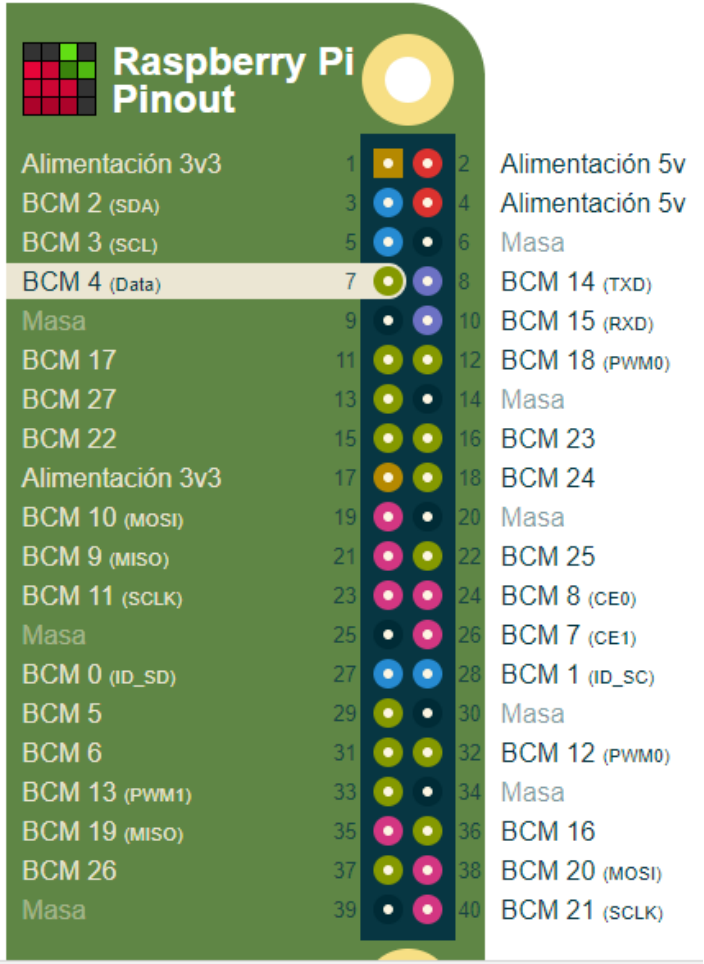
1-Wire

Descripción

El protocolo I2C permite que múltiples dispositivos "esclavos" se comuniquen con uno o más dispositivos "maestros“, con el uso de 2 hilos.

1-Wire

Pines


Raspberry Pi
Pinout
Alimentación 3v3 1
2 Alimentación 5v
BCM 2 (SDA) 3
4 Alimentación 5v
BCM 3 (SCL) 5
6 Masa
BCM 4 (Data) 7
8 BCM 14 (TXD)
Masa 9
10 BCM 15 (RXD)
BCM 17 11
12 BCM 18 (PWM0)
BCM 27 13
14 Masa
BCM 22 15
16 BCM 23
Alimentación 3v3 17
18 BCM 24
BCM 10 (MOSI) 19
20 Masa
BCM 9 (MISO) 21
22 BCM 25
BCM 11 (SCLK) 23
24 BCM 8 (CE0)
Masa 25
26 BCM 7 (CE1)
BCM 0 (ID_SD) 27
28 BCM 1 (ID_SC)
BCM 5 29
30 Masa
BCM 6 31
32 BCM 12 (PWM0)
BCM 13 (PWM1) 33
34 Masa
BCM 19 (MISO) 35
36 BCM 16
BCM 26 37
38 BCM 20 (MOSI)
Masa 39
40 BCM 21 (SCLK)


python
+
= Raspbian

1-Wire

Habilitar

El protocolo I2C permite que múltiples dispositivos "esclavos" se comuniquen con uno o más dispositivos "maestros“, con el uso de 2 hilos.

1-Wire

Ejemplos

El protocolo I2C permite que múltiples dispositivos "esclavos" se comuniquen con uno o más dispositivos "maestros“, con el uso de 2 hilos.

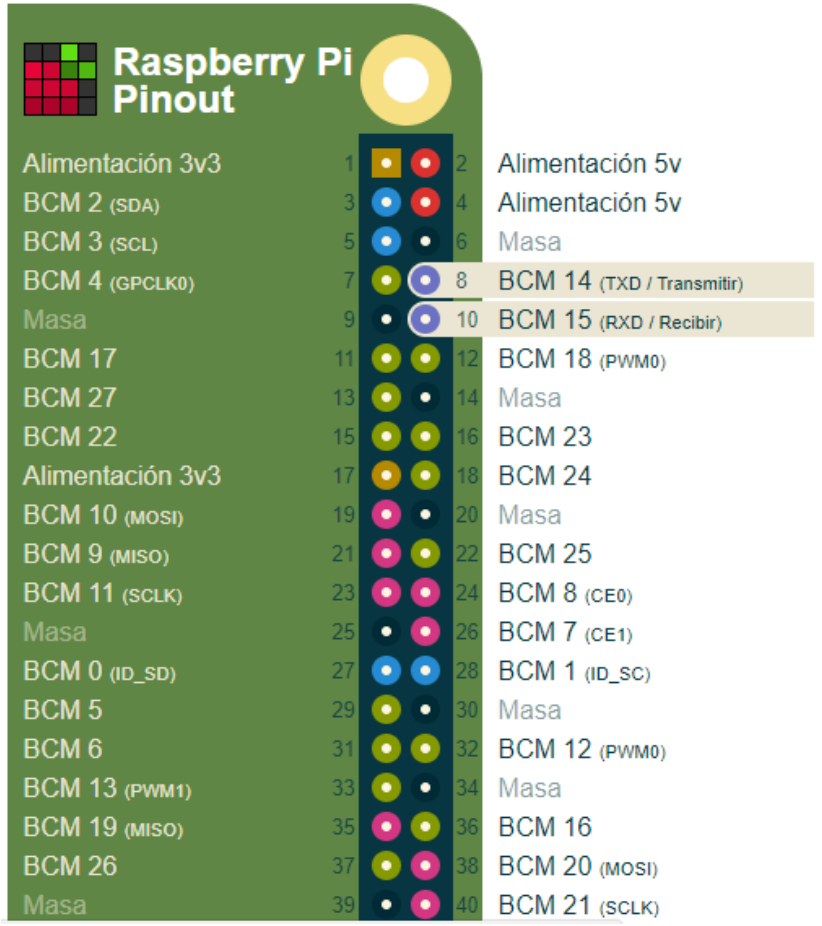
UART

Descripción

El protocolo I2C permite que múltiples dispositivos "esclavos" se comuniquen con uno o más dispositivos "maestros“, con el uso de 2 hilos.

UART

Pines



python + = Raspbian

UART

Habilitar

El protocolo I2C permite que múltiples dispositivos "esclavos" se comuniquen con uno o más dispositivos "maestros“, con el uso de 2 hilos.

UART

Ejemplos

El protocolo I2C permite que múltiples dispositivos "esclavos" se comuniquen con uno o más dispositivos "maestros“, con el uso de 2 hilos.